

中山大学计算机学院

人工智能

本科生实验报告

课程名称: Artificial Intelligence

学号	22336203	姓名	沈苇杭
----	----------	----	-----

一、实验题目

购房预测分类任务：利用感知机学习算法，在给定的数据集中完成购房预测分类任务。要求选择合适的损失函数，利用训练集完成网络训练，画出数据可视化图、loss 曲线图，计算模型收敛后的分类准确率。

二、实验内容

1. 算法原理

数据处理：采用归一化方式处理数据。归一化公式为：

$$x^* = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

利用这种方式，可以把数据放缩到[0,1]区间，使得各个特征对目标的影响一致。

感知机学习算法：

- 设置步长 η ，初始化权重 w 和偏置 b 。可以随机初始化，也可以初始化为特定的值。
- 利用公式 $y(x)=\text{sign}(wx+b)$ 计算预测值。
- 利用公式 $-y_i(w \cdot x_i + b)$ 计算单个数据点的 cost 值，若 cost 值大于 0，那么这个点是误分类点。
- 利用公式 $L(w,b)=-\sum_{x_i \in M} y_i(w \cdot x_i + b)$ 计算损失函数，其中 M 是误分类点的集合。
- 选择一个误分类点，利用公式 $w = w + \eta y_i x_i$ 和 $b = b + \eta y_i$ 更新权重和偏置。

2. 关键代码展示

a) 数据处理部分：

```

data = pd.read_csv("data.csv", encoding="utf-8")
data = pd.DataFrame(data)
X = data.values[:, :-1] # 前两列数据
detail = X.shape
row = detail[0] # 行数
col = detail[1] # 列数
min_values = np.min(X, axis=0)
max_values = np.max(X, axis=0)
X_fixed = np.zeros((row, col)) # 存储归一化后的数据，初始化为全 0
for i in range(0, 400): # 这个循环是归一化过程
    #print(X[i])
    X_fixed[i][0] = round((X[i][0] - min_values[0]) / (max_values[0] -
min_values[0]), 6)
    X_fixed[i][1] = round((X[i][1] - min_values[1]) / (max_values[1] - min_values[1]),
6)
    #print(X[i][0] - min_values[0], max_values[0] - min_values[0], X[i][1] -
min_values[1], max_values[1] - min_values[1], X_fixed[i][0], X_fixed[i][1], X_fixed[i])
Y = data.values[:, -1] # 标准的分类结果
for i in range(400):
    if Y[i] == 0:
        Y[i] = -1 # 将标签 0 改成-1. 方便后续运算
Z = np.zeros(row) # 算法得出的分类结果

```

这一段是数据处理部分。先导入、分割数据集，然后对特征向量进行归一化，再修改部分标签以方便后续运算。

b) 计算预测值部分：

```

def sign(x):
    if x >= 0:
        return 1
    else:
        return -1

def Predict(X_fixed, Z, w, b, index):
    Z[index] = sign(np.dot(X_fixed[index], w) + b)
    return Z[index]

```

使用原理部分给出的公式计算预测值。

c) 计算单个数据点的 cost 值部分：

```

def Cost(X_fixed, Y, index, w, b): # 单个数据点的损失
    linear_com = np.dot(w, X_fixed[index]) + b
    cost = -1 * Y[index] * linear_com # - yi * (w * xi + b)
    #print(w, X[index], Y[index], linear_com, cost)
    return cost

```

d) 计算损失函数部分：

```

def Loss(X_fixed, Y, row, w, b): # 求损失函数
    sum = 0

```

```

count = 0
for i in range(0, row):
    if Cost(X_fixed, Y, i, w, b) >= 0:
        sum = sum + Cost(X_fixed, Y, i, w, b)
        count = count + 1 # 误分类的总数
return sum / count

```

e) 更新权重和偏置部分:

```

def Gradient_Descent(X_fixed, Y, w, b, n, row):
    for index in range(0, row):
        #print(Y[index])
        if Cost(X_fixed, Y, index, w, b) >= 0: # 误分类点, 更新参数
            b = b + n * Y[index]
            w = w + n * np.dot(Y[index], X_fixed[index])
            #print(n, Y[index], np.dot(Y[index], X[index]), X[index])
    return w, b

```

f) 迭代部分:

```

def Iterate(time, X_fixed, Y, Z, w, b, n, row, loss_list): # 迭代
    for i in range(0, time):
        loss_list.append(Loss(X_fixed, Y, row, w, b))
        w, b = Gradient_Descent(X_fixed, Y, w, b, n, row) # 更新所有参数
    sum = 0
    for i in range(0, row):
        tmp = Predict(X_fixed, Z, w, b, i)
        if tmp >= 0 and Y[i] == 1:
            sum = sum + 1
        elif tmp < 0 and Y[i] == -1:
            sum = sum + 1
    print("正确率: ", sum / row)
    return loss_list, w, b

```

在这个部分, 首先进行 time 次数的迭代, 每次迭代都将新的损失函数值加入损失函数值的列表 (用于绘图), 并返回更新后的权重和偏置。然后, 遍历所有数据点, 统计出分类正确的点的总数, 计算正确率, 返回损失函数列表、权重, 偏置。

g) 绘图部分:

```

x_line = np.linspace(min(X_fixed[:, 0]), max(X_fixed[:, 0]), 100)
y_line = -1 * w[0]/w[1] * x_line - b / w[1]
# 绘制散点图
figure1 = plt.figure(1)
for i in range(row):
    if Y[i] == -1:
        plt.scatter(X_fixed[i][0], X_fixed[i][1], color='purple')
    else:
        plt.scatter(X_fixed[i][0], X_fixed[i][1], color='yellow')
# 绘制直线的图像

```

```
plt.plot(x_line, y_line, color='black', label='Linear Function')

# 标题, x 轴, y 轴
plt.xlabel('Age')
plt.ylabel('Money')
fig2 = plt.figure(2)
y = loss_list
x = list(range(len(y)))
plt.scatter(x, y, marker='.', color='red')
plt.xlabel("Time")
plt.ylabel("Loss")
# 显示
plt.show()
```

在这个部分，绘制两张图，一张是数据可视化图，另一张是损失函数图。

3. 优化和创新

第一次运行时，将迭代次数设置为 1000，此时正确率为 0.7675

第二次运行，将迭代次数设置为 2000，此时正确率为 0.7825

第三次运行，将迭代次数设置为 3000，此时正确率为 0.795

第四次运行，将迭代次数设置为 4000，此时正确率为 0.79

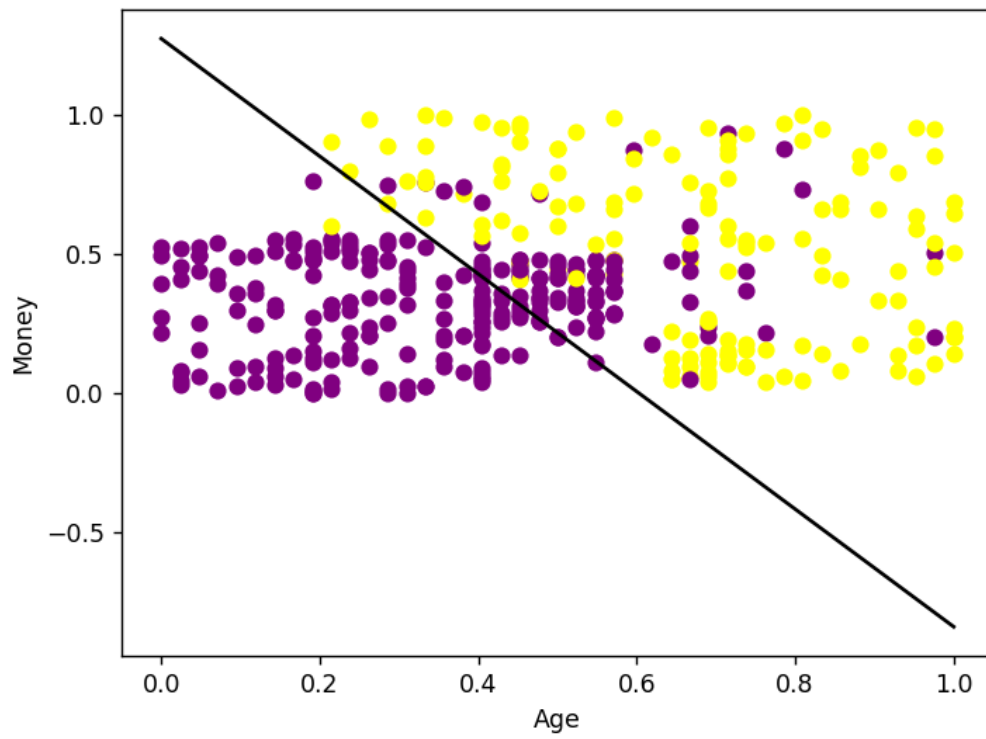
因此最终将迭代次数设置为 3000.

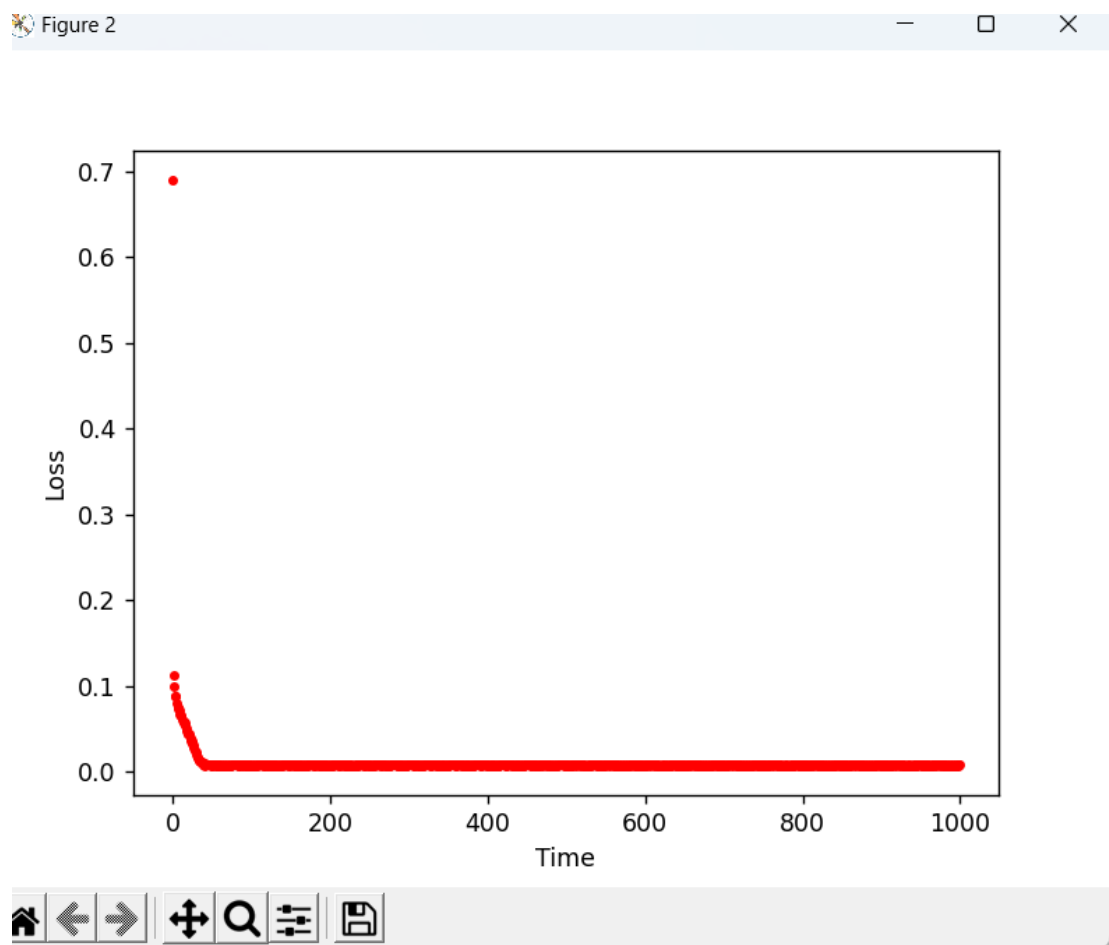
三、实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

迭代 1000 次：

Figure 1





迭代 2000 次:

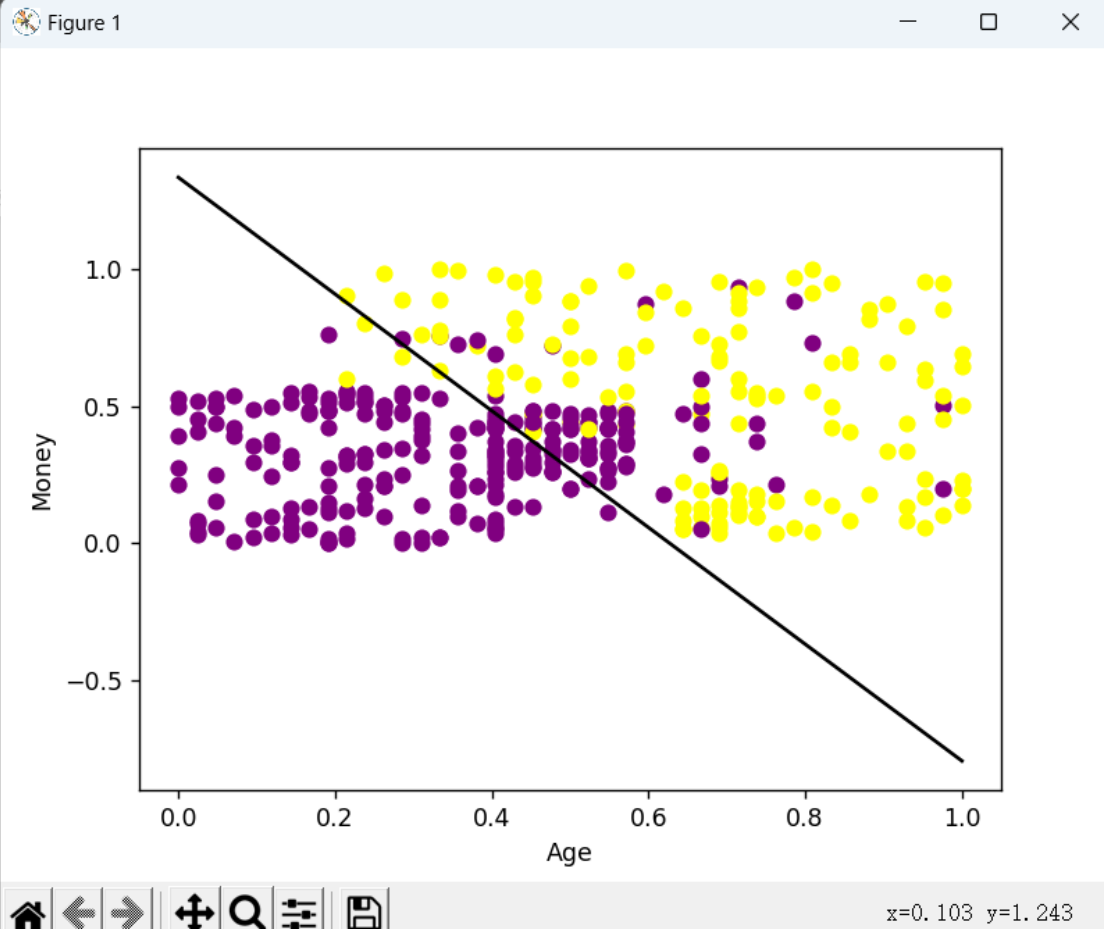
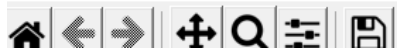
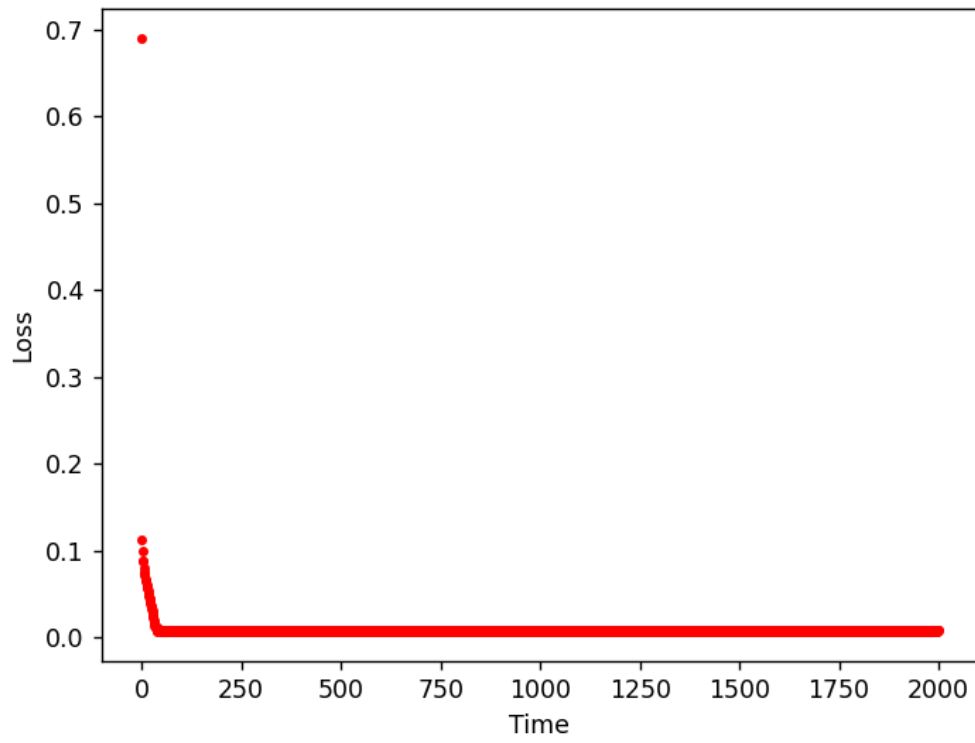
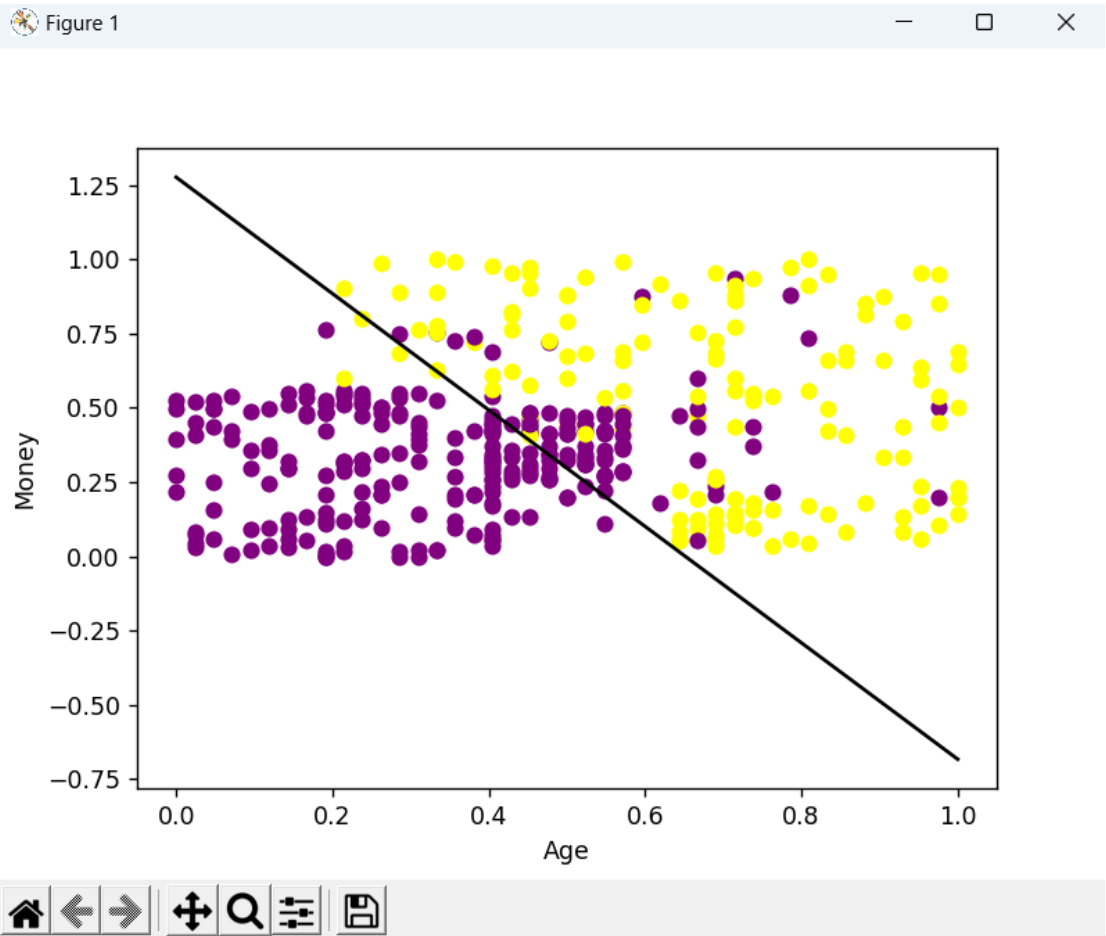
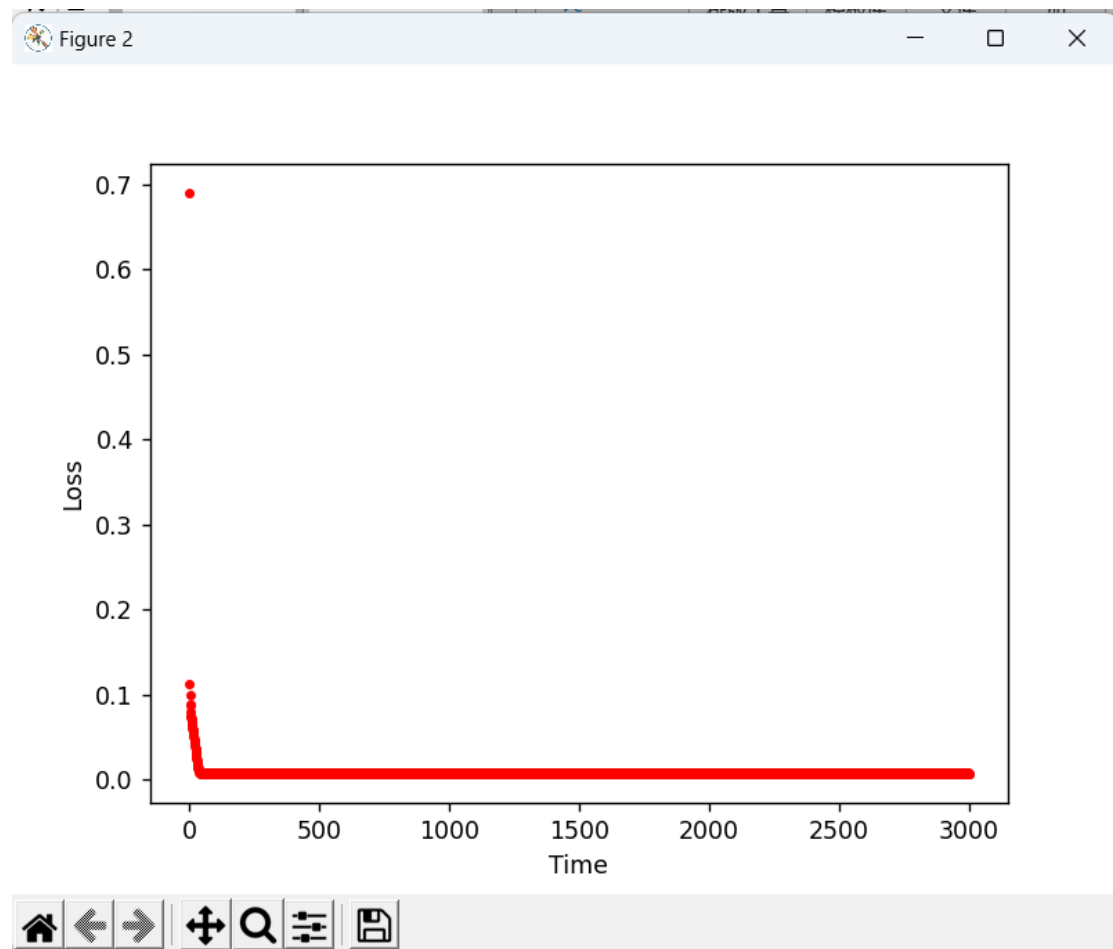


Figure 2

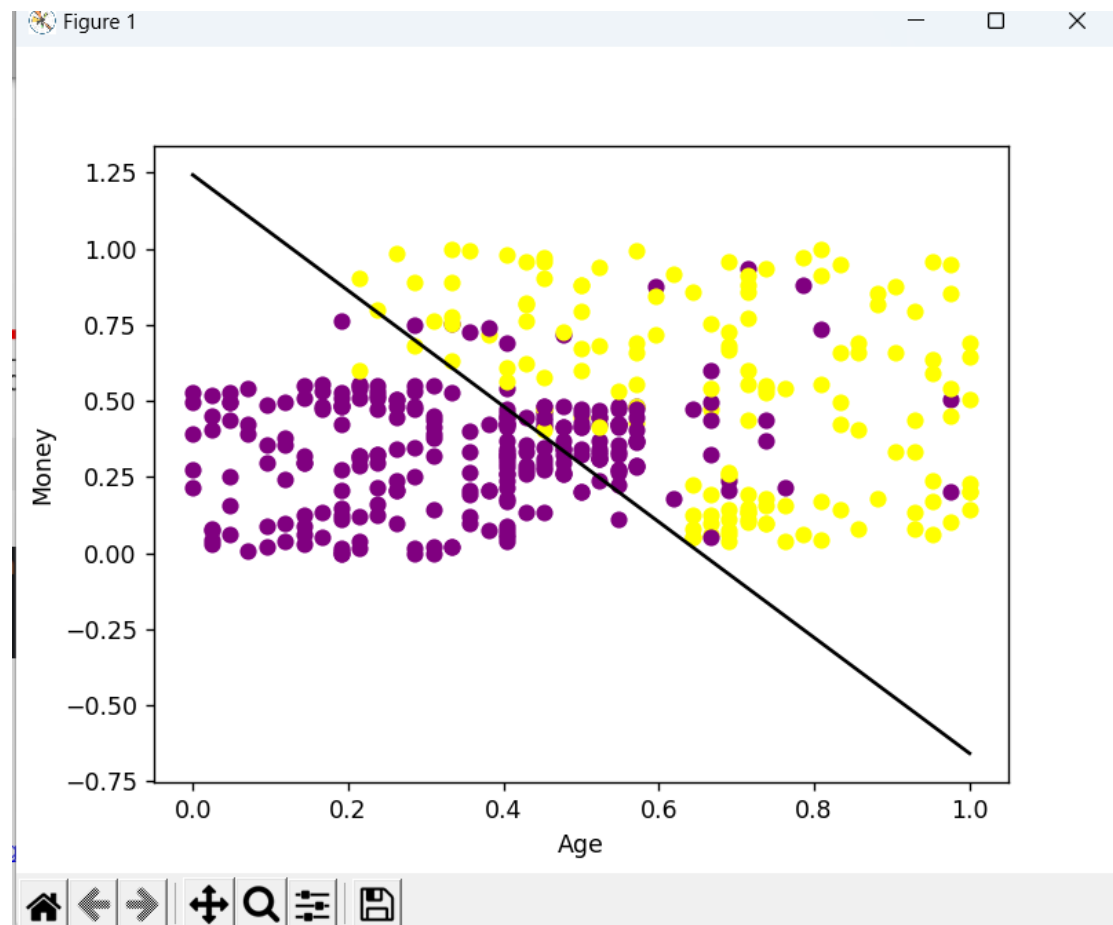


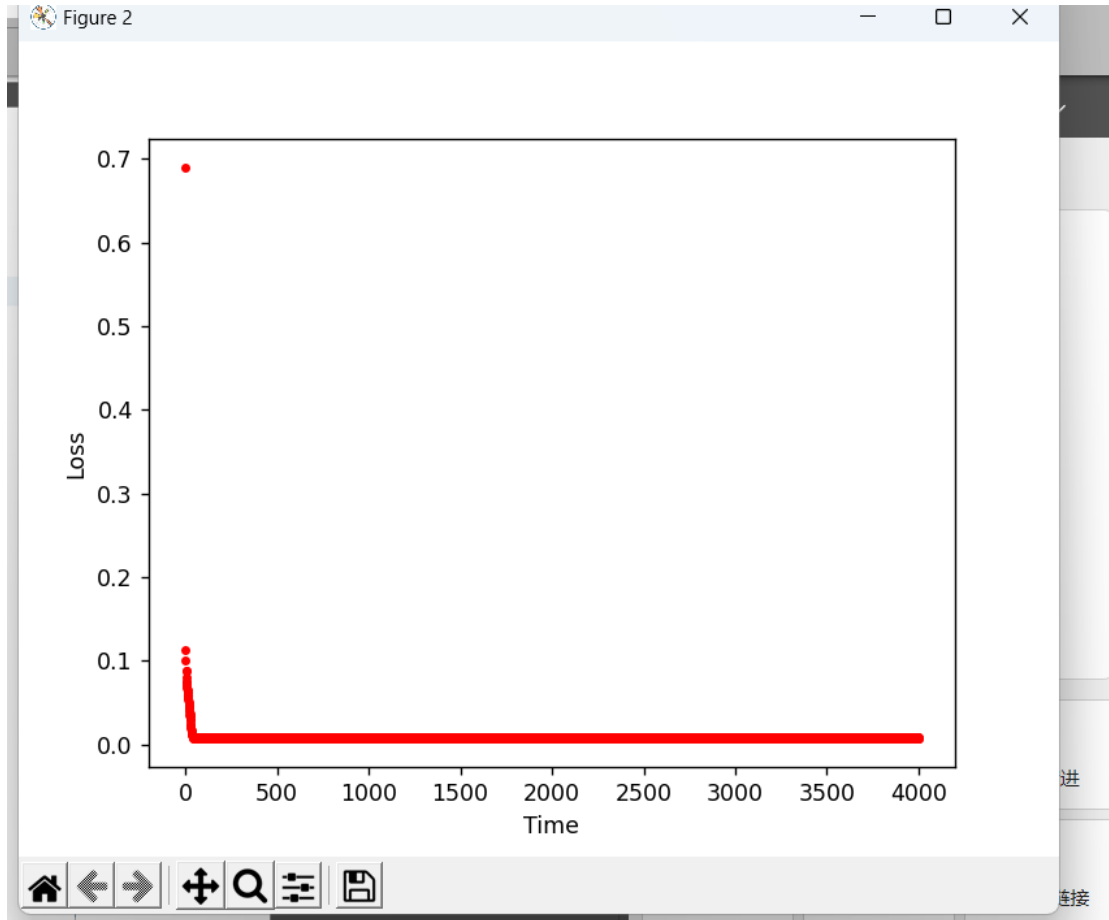
迭代 3000 次:





迭代 4000 次：





2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）
迭代 1000 次：

```
E:\学习\Pycharm\AILab7-1\venv\Scripts\python.exe  
正确率： 0.7675
```

迭代 2000 次：

```
E:\学习\Pycharm\AILab7-1\venv\Scripts\python.exe  
正确率： 0.7825
```

迭代 3000 次：

```
E:\学习\Pycharm\AILab7-1\venv\Scripts\python.exe E  
正确率： 0.795
```

迭代 4000 次:

```
E:\学习\Pycharm\AILab7-1\venv\Scripts\python.exe  
正确率: 0.79
```

四、 参考资料

1. [项目文件预览 - perceptron-algorithm - GitCode](#)
2. [my_Machine_learning/perception_code.py at master · zilumeng/my_Machine_learning \(github.com\)](#)
3. [python 之 numpy 的基本使用-CSDN 博客](#)
4. [python 如何使用 Matplotlib 画图（基础篇） - 知乎 \(zhihu.com\)](#)
5. 理论课 PDF